Mobile Application Security Assessment

DIVA Android

Damn Insecure and Vulnerable App — six challenges

Prepared by	Rahaf Albarakati
Target	DIVA (package `jakhar.aseem.diva`) on an Android emulator
Test type	Static and dynamic analysis
Tools	Android Studio Logcat, ADB, jadx
Classification	OWASP Mobile Top 10 (2024)

> DIVA is an intentionally vulnerable Android application built for learning mobile application security. All testing was performed in a controlled emulator environment, and no real-world application was targeted. All data shown (card numbers, credentials) is test data.

Executive Summary

Six DIVA challenges were assessed. The most significant issues are sensitive data written to system logs and a secret key hardcoded in the application, both of which expose data that should never leave the user or the backend. Most issues stem from the app trusting the device and the user input instead of enforcing protection itself.

#	Issue	OWASP Mobile 2024	Severity
1	Insecure Logging	M9 Insecure Data Storage · M6	High
2	Hardcoding Issues (Part 1)	M1 Improper Credential Usage · M7	High
3	Insecure Data Storage (Part 1)	M9 Insecure Data Storage	Medium
4	Input Validation Issues (Part 1)	M4 Insufficient Input/Output Validation	High
5	Input Validation Issues (Part 2)	M4 · M8 Security Misconfiguration	Medium
6	Access Control Issues (Part 1)	M3 Insecure Authentication/Authorization	Medium

Severity Rating Methodology

Each issue is rated on two separate axes, and the final rating is derived from both:

Axis	What it measures
Ease of exploitation	Does the attacker need physical access, ADB, root, or a malicious app on the device? Are free tools enough?
Impact	Which data or features are exposed? Payment data, credentials, or only app behavior?

Finding 1

High

Insecure Logging

Component	Challenge 1 — credit card checkout screen
Classification	OWASP M9: Insecure Data Storage · M6: Inadequate Privacy Controls · CWE-532

When checkout fails, the app writes the full credit card number to Logcat, where anyone with access to the device logs can read it.

Description

Android's Logcat is meant for debugging output. In this challenge, the app logs the card number entered by the user in plain text. The app also does not validate the card number before processing it.

Evidence

A test card number `123456788` was entered and submitted. Filtering Logcat by the app's tag shows the full number in an error-level entry:

```
# Logcat — process jakhar.aseem.diva, tag diva-log
E  diva-log  Error while processing transaction with credit card: 123456788
```

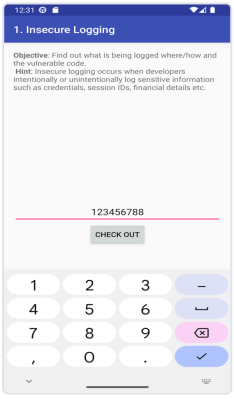


Figure 1: Test card number entered

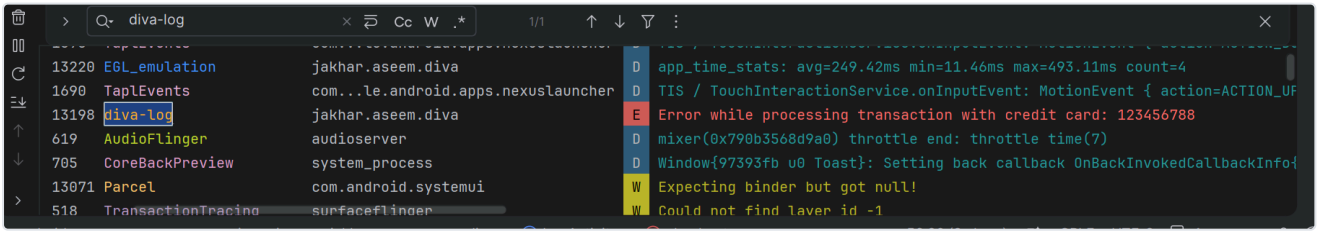


Figure 2: The card number appears in Logcat under the `diva-log` tag

Severity Rating

Ease of exploitation	Impact	Final rating
Medium	**High**	**High**
Requires access to device logs (ADB, a connected computer, or a privileged app).	Full payment card number exposed in plain text, enabling fraud.	Payment data exposure outweighs the access requirement.

Remediation

- Never log sensitive data such as card numbers, passwords, tokens, or session IDs.
- Remove or disable debug and verbose logging in release builds.
- If logging is necessary, mask the data (for example, show only the last four digits).
- Validate the card number format before processing.

```
// Insecure
Log.e("diva-log", "Error while processing transaction with credit card: " + cardNumber);

// Secure
Log.e("diva-log", "Error while processing transaction");
```

Finding 2

High

Hardcoding Issues (Part 1)

Component	Challenge 2 — vendor key access screen
Classification	OWASP M1: Improper Credential Usage · M7: Insufficient Binary Protections · CWE-798

The vendor secret key is written directly in the app's code. Decompiling the APK reveals it, and entering it grants access to the protected feature.

Description

Anything compiled into an APK can be recovered. In this challenge, decompiling the app with jadx reveals the key that the access check compares against.

Evidence

The key recovered from the decompiled code, `vendorsecretkey`, was entered in the app, which responded with “Access granted!”.

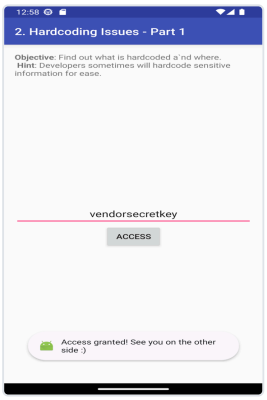


Figure 3: Hardcoded key accepted — access granted

Severity Rating

Ease of exploitation	Impact	Final rating
High	**Medium**	**High**
Only the APK and a free decompiler are needed, with no device access.	Unlocks a protected feature; the same key is identical for every user.	Trivial to extract, and cannot be fixed without releasing a new app version.

Remediation

- Do not hardcode passwords, API keys, or tokens in the application.
- Perform access checks on a backend server instead of inside the app.
- When local storage of secrets is unavoidable, use the Android Keystore or EncryptedSharedPreferences.
- Apply code obfuscation (R8/ProGuard) to slow down reverse engineering — as an extra layer, not a fix.

Finding 3

Medium

Insecure Data Storage (Part 1)

Component	Challenge 3 — credential save screen
Classification	OWASP M9: Insecure Data Storage · CWE-312

The app saves the username and password in SharedPreferences without encryption, so they can be read directly from the app's storage.

Description

SharedPreferences are stored as a plain XML file in the app's data directory. Anyone who can access that directory (a rooted device, ADB on a debuggable app, or a backup) can read the saved credentials.

Evidence

Test credentials were entered and saved through the challenge screen.



Figure 4: Test credentials saved by the app

Severity Rating

Ease of exploitation	Impact	Final rating
Low	**High**	**Medium**
Requires root, a debuggable build, or access to a device backup.	User credentials exposed in plain text, possibly reused on other services.	High impact, limited by the access required.

Remediation

- Do not store passwords, PINs, or tokens in plain text.
- Use EncryptedSharedPreferences, with keys protected by the Android Keystore.
- Prefer keeping sensitive data on a secure backend rather than on the device.
- Set `android:allowBackup="false"` for apps that hold sensitive data.

Finding 4

High

Input Validation Issues (Part 1)

Component	Challenge 7 — user search
Classification	OWASP M4: Insufficient Input/Output Validation · CWE-89

The search field passes input to the local database query without validation, so a crafted search can return the data of all users instead of one.

Description

The challenge goal is to output the data of all three stored users with a single search, without knowing any username. Because the input is not filtered, special characters change the meaning of the database query.

Evidence

Searching for the known user `admin` returns that user's record, including the password and credit card number, which confirms the search reads sensitive fields from the database.

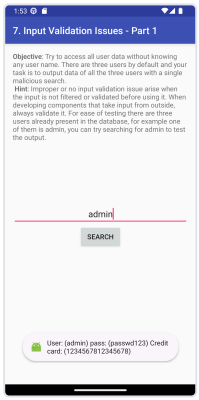


Figure 5: Baseline search for “admin” returns the full record

Severity Rating

Ease of exploitation	Impact	Final rating
High	**High**	**High**
Typed directly into the app’s search field with no extra tools.	Exposes every user’s password and card number.	Easy to trigger, with full data exposure.

Remediation

- Use parameterized queries (? placeholders) instead of building SQL strings from input.
- Validate input against an allow-list of expected characters and length.
- Do not store passwords or full card numbers in the local database.

Finding 5

Medium

Input Validation Issues (Part 2)

Component	Challenge 8 — URL viewer (WebView)
Classification	OWASP M4: Insufficient Input/Output Validation · M8: Security Misconfiguration · CWE-20

The app loads any URL the user enters in a WebView without checking the scheme or destination.

Description

The challenge accepts a URL and displays it inside the app. Without validation, the WebView may also accept schemes such as `file://`, which can be abused to read local files that the app has access to.

Evidence

An external URL (`https://www.google.com`) was entered and loaded inside the app with no restriction.

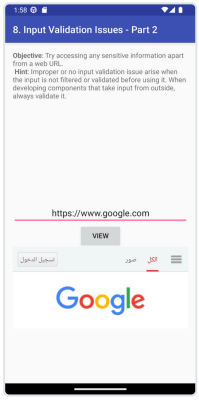


Figure 6: Arbitrary URL loaded in the app's WebView

Severity Rating

Ease of exploitation	Impact	Final rating
High	**Medium**	**Medium**
Any value can be typed into the URL field.	Possible local file disclosure or phishing content inside a trusted app.	Easy to trigger; impact depends on what the WebView can reach.

Remediation

- Allow only `https://` URLs from an allow-list of trusted domains.
- Disable file access in the WebView: `setAllowFileAccess(false)`.
- Disable JavaScript in the WebView unless it is required.

Finding 6

Medium

Access Control Issues (Part 1)

Component	Challenge 9 — Vendor API Credentials activity
Classification	OWASP M3: Insecure Authentication/Authorization · CWE-926

The activity that displays the vendor API credentials is exported, so it can be opened directly from outside the app without going through the intended flow.

Description

Exported Android components can be launched by other apps or by ADB. Since this activity performs no check of its own, anyone who launches it directly sees the credentials.

Evidence

The activity displays the API key, username, and password in plain text.

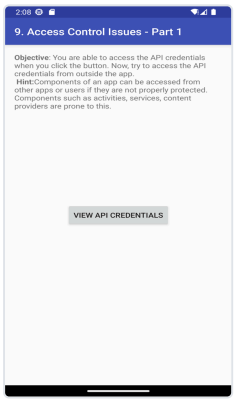


Figure 7: Challenge screen with the “View API Credentials” button

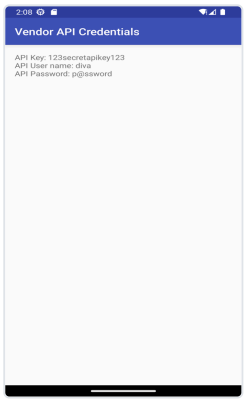


Figure 8: API credentials displayed by the activity

Severity Rating

Ease of exploitation	Impact	Final rating
Medium	**Medium**	**Medium**
Requires ADB or a malicious app installed on the same device.	Exposes the vendor API key and its credentials.	Real credential exposure, limited by the on-device access needed.

Remediation

- Set `android:exported="false"` for components that do not need to be reachable from outside.
- Check authorization inside the activity itself, not only in the screen that leads to it.
- Protect required exported components with a signature-level permission.
- Do not display or store API credentials in the client app.